Sistema de Planilla Hospitalaria

Documento explicativo del diagrama de clases UML

Curso	**EIF201 Programación I · Universidad Nacional, Sección Regional Central Occidente**
Proyecto	Proyecto de Programación #2 · Sistema de Planilla Hospitalaria
Estudiante	Marco Jiménez Acosta
Cédula	120100776
Modalidad	Proyecto individual (autorizado por el profesor)

> El diagrama completo está en la **página siguiente**. El archivo editable es `Diagrama de clases - Planilla Hospitalaria.drawio`.

1. Qué hace el sistema

Es una aplicación de consola en C++ que administra la planilla de un hospital:

- registra departamentos y sus puestos;
- registra empleados de tres tipos: médicos, enfermeros y administrativos;
- calcula el salario bruto y el neto con horas extra, bonificaciones y deducciones de ley;
- procesa los pagos.

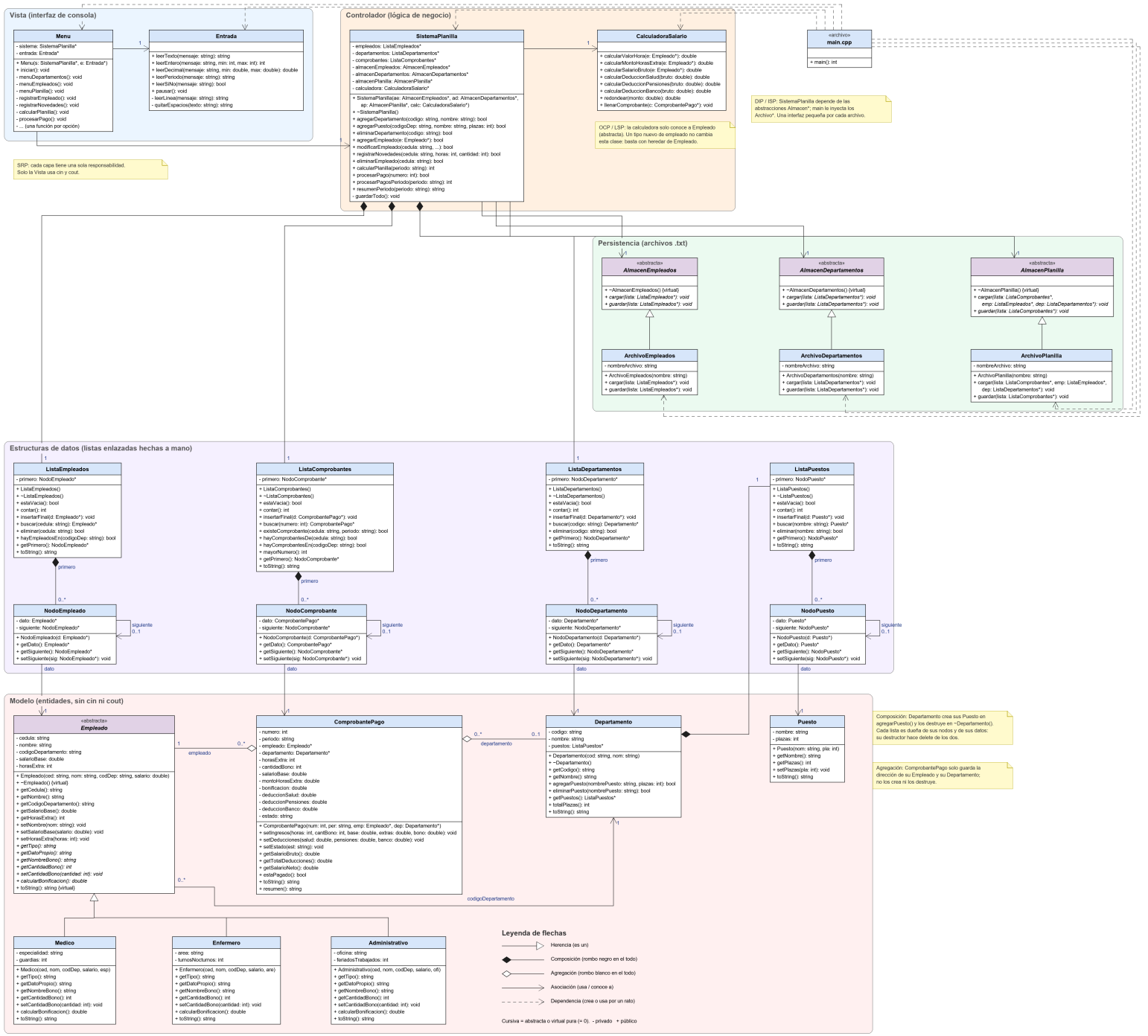
Toda la información se guarda en tres archivos de texto: `empleados.txt`, `departamentos.txt` y `planilla.txt`. Se cargan solos al iniciar y se actualizan con cada cambio.

2. Arquitectura por capas

El diagrama está dividido en cinco paquetes. Cada uno tiene una sola responsabilidad:

Capa	Clases	Responsabilidad
Vista	`Menu`, `Entrada`	Hablar con el usuario: mostrar menús y validar lo que escribe. Es la única capa que usa `cin` y `cout`.
Controlador	`SistemaPlanilla`, `CalculadoraSalario`	La lógica del negocio: registrar, calcular la planilla, procesar pagos y aplicar las reglas salariales.
Persistencia	`AlmacenEmpleados`, `AlmacenDepartamentos`, `AlmacenPlanilla` (abstractas) y `ArchivoEmpleados`, `ArchivoDepartamentos`, `ArchivoPlanilla`	Cargar y guardar los datos en archivos de texto.

Diagrama de clases UML · Sistema de Planilla Hospitalaria



Sistema de Planilla Hospitalaria — Diagrama de clases UML

Estructuras de datos	`ListaEmpleados`, `ListaDepartamentos`, `ListaPuestos`, `ListaComprobantes` y sus nodos	Guardar los datos en memoria con listas enlazadas simples hechas a mano.
Modelo	`Empleado` (abstracta), `Medico`, `Enfermero`, `Administrativo`, `Departamento`, `Puesto`, `ComprobantePago`	Las entidades del hospital: sus datos y sus reglas propias, sin entrada ni salida por consola.

3. Relaciones entre clases

3.1 Herencia

`Empleado` es una **clase abstracta**: tiene métodos virtuales puros (`getTipo`, `getDatoPropio`, `getNombreBono`, `getCantidadBono`, `setCantidadBono`, `calcularBonificacion`), así que no se pueden crear objetos `Empleado` directamente. `Medico`, `Enfermero` y `Administrativo` heredan de ella de forma pública y redefinen esos métodos:

Clase	Dato propio	Bonificación
`Medico`	especialidad	guardias × ₡35 000
`Enfermero`	área	turnos nocturnos × ₡15 000
`Administrativo`	oficina	feriados trabajados × (salario base / 30)

El destructor de `Empleado` es **virtual**. Así, al hacer `delete` con un `Empleado*`, se ejecuta primero el destructor de la clase hija y luego el de la madre.

En la persistencia también hay herencia: cada `Archivo...` hereda de su `Almacen...` abstracto e implementa `cargar` y `guardar`.

3.2 Composición

Todo	Parte	Cómo se ve en el código
`Departamento`	`ListaPuestos` → `Puesto`	El departamento crea su lista en el constructor y crea cada `Puesto` con `new` dentro de `agregarPuesto()`. En `~Departamento()` hace `delete puestos`, y la lista borra cada puesto. Un puesto no existe fuera de su departamento.
`SistemaPlanilla`	las 3 listas	Las crea en su constructor y las destruye en su destructor.
Cada lista	sus nodos	La lista crea cada nodo al insertar y lo destruye al eliminar o en su destructor.

3.3 Agregación

`ComprobantePago` guarda punteros a un `Empleado` y a un `Departamento` que ya existen. **No los crea ni los destruye**: su destructor está vacío. Si se borra un comprobante, el empleado y el departamento siguen existiendo en el sistema.

Multiplicidad: un empleado puede tener muchos comprobantes (`1` — `0..*`), uno por periodo.

Por esta relación el sistema **no deja eliminar** a un empleado o un departamento que tenga comprobantes: esos comprobantes quedarían apuntando a memoria liberada (un puntero colgante).

3.4 Asociación

Clase	Usa a	Para qué
`Menu`	`SistemaPlanilla`	Pedirle cada operación
`Menu`	`Entrada`	Leer y validar los datos que escribe el usuario
`SistemaPlanilla`	`CalculadoraSalario`	Calcular los montos de cada comprobante
`SistemaPlanilla`	`AlmacenEmpleados`, `AlmacenDepartamentos`, `AlmacenPlanilla`	Cargar al iniciar y guardar después de cada cambio
Cada nodo	su dato y el nodo siguiente	Formar la lista enlazada
`Empleado`	`Departamento`	Saber a qué departamento pertenece. Guarda `codigoDepartamento`, y con ese dato se encuentra el departamento

Estas clases se conocen por medio de punteros que llegan por el constructor, salvo `Empleado`, que conoce a su departamento por el código. Ninguna de ellas destruye a la otra.

Multiplicidad de `Empleado` y `Departamento`: cada empleado pertenece a un departamento (`1`), y un departamento puede tener muchos empleados (`0..*`).

3.5 Dependencia

`main.cpp` **crea** los objetos concretos (`ArchivoEmpleados`, `ArchivoDepartamentos`, `ArchivoPlanilla`, `CalculadoraSalario`, `SistemaPlanilla`, `Entrada` y `Menu`), se los pasa a quien los necesita y al final los libera en orden inverso.

4. Principios SOLID en el diseño

Principio	Dónde se ve
S — Responsabilidad única	Cada capa hace una sola cosa. Por ejemplo, `Empleado` no sabe guardar en archivos (de eso se encarga `ArchivoEmpleados`) ni mostrar menús (de eso se encarga `Menu`).
O — Abierto/cerrado	`CalculadoraSalario` solo conoce a `Empleado`. Para agregar un nuevo tipo de empleado o especialidad se crea una clase hija con su propia `calcularBonificacion()`, sin modificar la lógica salarial central.
L — Sustitución de Liskov	`SistemaPlanilla::calcularPlanilla` recorre la lista tratando a todos como `Empleado*`. Un `Medico`, un `Enfermero` o un `Administrativo` funcionan igual en ese lugar, y cada uno aplica su propia regla.

I — Segregación de interfaces	En vez de una sola interfaz grande de «guardar todo», hay tres pequeñas (`AlmacenEmpleados`, `AlmacenDepartamentos`, `AlmacenPlanilla`), cada una con solo `cargar` y `guardar` de lo suyo.
D — Inversión de dependencias	`SistemaPlanilla` depende de las abstracciones `Almacen...`, no de los archivos. `main` le **inyecta** las implementaciones concretas por el constructor. Cambiar los `.txt` por otra forma de guardar no obliga a tocar el sistema.

5. Reglas de cálculo

Concepto	Regla
Valor de la hora	salario base / 240
Hora extra	valor de la hora × 1,5
Salario bruto	base + horas extra + bonificación
Deducciones de ley (sobre el bruto)	CCSS Salud 5,50 % · CCSS Pensiones (IVM) 4,33 % · Banco Popular 1 %
Salario neto	bruto − deducciones
Redondeo	todos los montos a 2 decimales (céntimos)

Ejemplo (médico): salario de ₡1 500 000, 10 horas extra y 3 guardias → horas extra ₡93 750, bonificación ₡105 000, bruto ₡1 698 750.

6. Correspondencia entre el diagrama y el código

- Cada clase del diagrama tiene su `.h` (con guardas `#ifndef`, `#define` y `#endif`) y su `.cpp`. Son 25 clases y `main.cpp`.
- Los atributos del diagrama son los mismos del `.h`, con la misma visibilidad.
- Los rombos negros corresponden a un `delete` en el destructor del todo. Los rombos blancos corresponden a punteros que nunca se borran en esa clase.
- Las clases del modelo no usan `cin` ni `cout`. Sus `toString()` devuelven texto armado con `stringstream`.